SQL & WEB SECURITY CHALLENGE

MARIA ROSARIO MAMANI POMA

24-10-2026

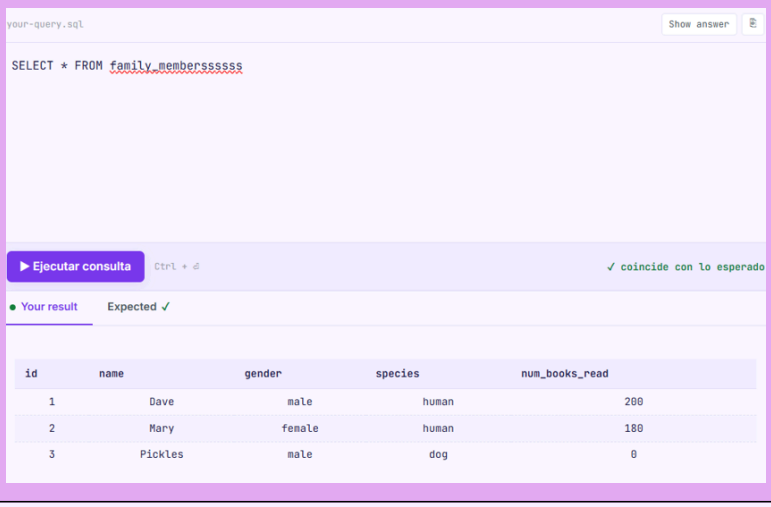
1 **Ejercicio 1: Seleccionar todos los registros**

2 **Objetivo del ejercicio:**
Obtener todos los datos de la tabla family_members, con todas sus columnas y filas.

3 **Concepto SQL:**
SELECT *, que sirve para traer todas las columnas de una tabla sin tener que escribirlas una por una.

4 **Consulta ejecutada:**
SELECT * FROM family_members;

5 **Resultado:**



6 **Interpretación:**
La consulta devuelve los tres miembros de la familia con todos sus datos. Dave y Mary son humanos y Pickles es un perro, que no ha leído libros.

7 **Aprendizaje:**
El * es un atajo útil para explorar una tabla nueva, pero en consultas reales conviene nombrar solo las columnas necesarias, sobre todo si la tabla es grande.

8 **Aplicación en el mundo real:**
En una organización sirve para revisar rápidamente el contenido de una tabla, por ejemplo, al auditar una base de datos de usuarios o de accesos. También ayuda a identificar campos sensibles que deberían protegerse o excluirse de los reportes.

1 **Ejercicio 2: Seleccionar columnas específicas**

2 **Objetivo del ejercicio:**
Obtener solo las columnas name y species de la tabla family_members, sin traer las demás.

3 **Concepto SQL:**
SELECT columna1, columna2: permite elegir qué columnas devolver, separándolas con comas.

4 **Consulta ejecutada:**
SELECT name, species FROM family_members;

5

Resultado:



6

Interpretación:
La consulta devuelve solo el nombre y la especie de cada miembro. Al omitir gender, num_books_read y las demás columnas, el resultado es más corto y enfocado.

7

Aprendizaje:
En lugar de *, se listan las columnas que se necesitan. Esto hace la consulta más clara y ligera, y es una buena práctica en consultas reales.

8

Aplicación en el mundo real:
En una auditoría, se puede pedir solo los campos relevantes para un reporte, por ejemplo nombres de usuarios y su rol o tipo de cuenta, sin exponer datos sensibles que no se necesitan. Esto respeta el principio de minimización de datos.

1

Ejercicio 3: Filtrar filas con WHERE

2

Objetivo del ejercicio:
Obtener solo las filas de la tabla family_members que corresponden a perros (species = 'dog').

3

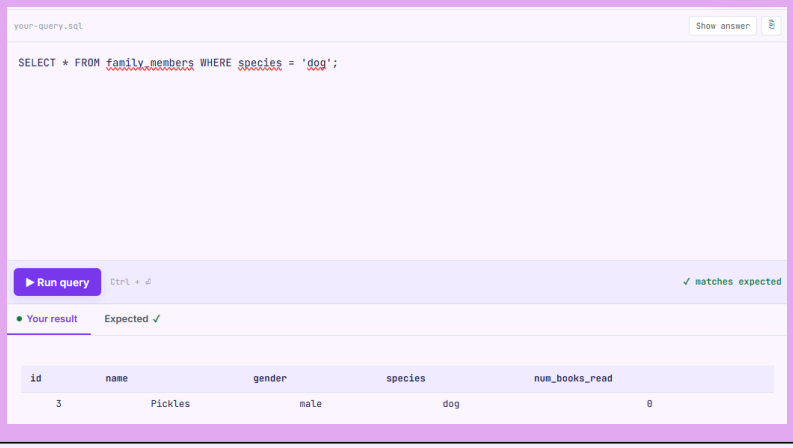
Concepto SQL:
WHERE: permite filtrar filas según una condición. El operador = compara el valor de una columna con un valor exacto, que en este caso es texto y va entre comillas simples.

4

Consulta ejecutada:
SELECT * FROM family_members WHERE species = 'dog';

5

Resultado:



6

Interpretación:
La consulta filtra la tabla y devuelve únicamente el registro cuya especie es perro. Dave y Mary, que son humanos, quedan excluidos.

7

Aprendizaje:
WHERE se coloca después de FROM y evalúa una condición para cada fila. Las comillas son obligatorias en valores de texto como 'dog', pero no en palabras clave como WHERE.

8

Aplicación en el mundo real:
En seguridad de la información, se puede usar WHERE para filtrar, por ejemplo, solo los usuarios con rol administrador (rol = 'admin') al revisar accesos, o solo los registros de un área específica durante una auditoría.

1

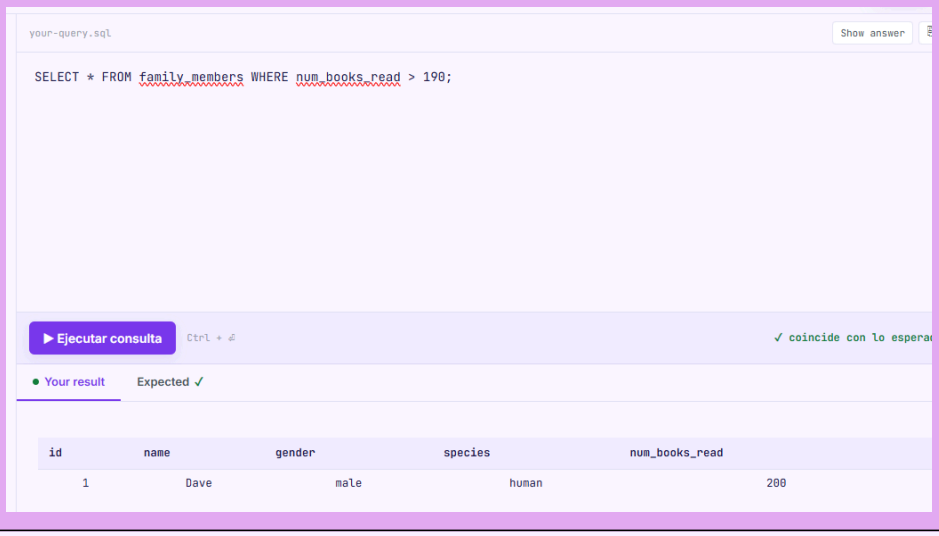
Ejercicio 4: WHERE ... Greater than (filtrar con mayor que)

2 **Objetivo del ejercicio:**
Obtener solo los miembros de family_members cuyo num_books_read es mayor que 190.

3 **Concepto SQL:**
WHERE con el operador >, que compara valores numéricos y devuelve las filas donde la condición se cumple.

4 **Consulta ejecutada:**
SELECT * FROM family_members WHERE num_books_read > 190;

5 **Resultado:**



6 **Interpretación:**
La consulta devuelve solo a Dave, porque es el único con más de 190 libros. Mary tiene 180 y Pickles tiene 0, así que quedan fuera.

7 **Aprendizaje:**
A diferencia de los textos, los números no llevan comillas. Además, > es solo uno de los operadores de comparación; también existen <, >=, <= y != (o <>, según el motor).

8 **Aplicación en el mundo real:**
En seguridad, puedes filtrar usuarios con un número de intentos fallidos mayor a un umbral, o cuentas con un número de accesos fuera de lo normal, para detectar posibles incidentes o revisar políticas de acceso.

---

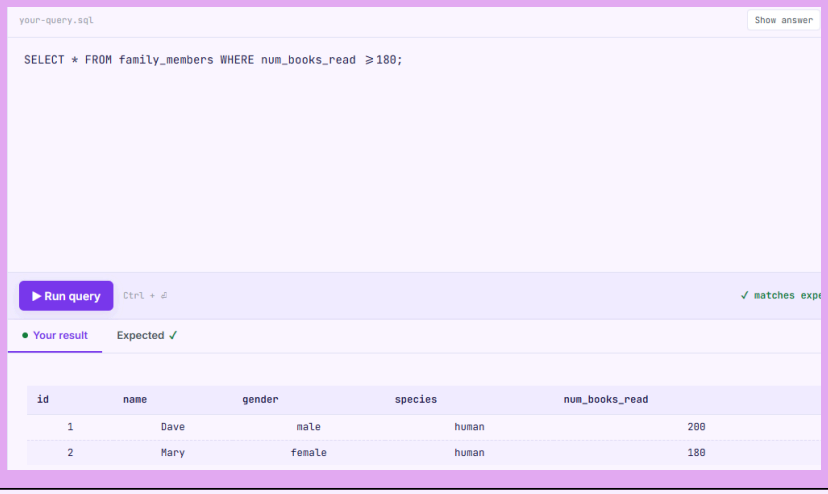
1 **Ejercicio 5: WHERE ... Greater than or equal (filtrar con mayor o igual que)**

2 **Objetivo del ejercicio:**
Obtener los miembros de family_members cuyo num_books_read es mayor o igual que 180.

3 **Concepto SQL:**
WHERE con el operador >=, que incluye el valor exacto además de los valores mayores.

4 **Consulta ejecutada:**
SELECT * FROM family_members WHERE num_books_read >= 180;

5 **Resultado:**



6

Interpretación:
La consulta devuelve a Dave, con 200 libros, y a Mary, con exactamente 180. Pickles, con 0, queda fuera.

7

Aprendizaje:
La diferencia con > está en el valor límite: > excluye el 180, mientras que >= lo incluye. Para escribir el operador, en un motor real como PostgreSQL o MySQL se usa >=. En tu consulta escribiste el símbolo «⩾», que la plataforma aceptó, pero en una base de datos real daría error.

8

Aplicación en el mundo real:
En seguridad, puedes usar >= para seleccionar cuentas que alcanzan o superan un umbral, por ejemplo usuarios con 5 o más intentos fallidos de inicio de sesión, o equipos cuya versión de software es igual o posterior a la requerida por una política.

1

Ejercicio 6: AND (combinar varias condiciones en WHERE)

2

Objetivo del ejercicio:
Obtener los amigos de Pickles que son perros y miden menos de 45 cm en la tabla friends_of_pickles.

3

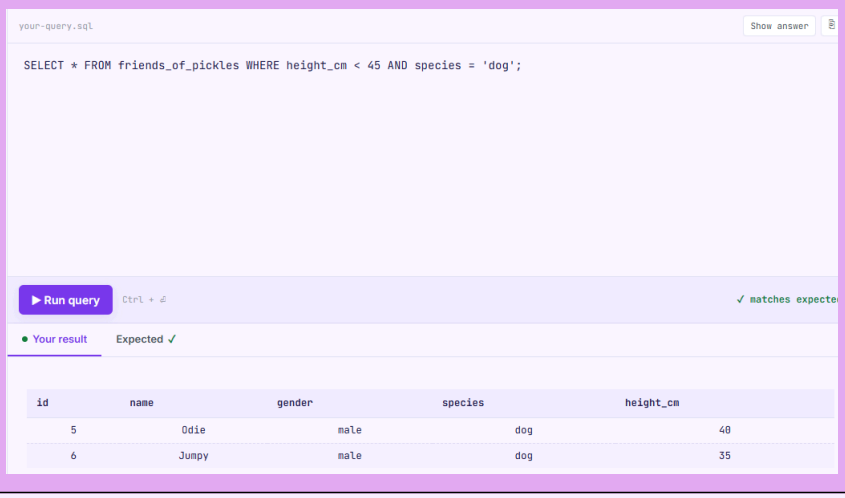
Concepto SQL:
AND:une dos o más condiciones en el WHERE, y una fila solo se devuelve si cumple todas.

4

Consulta ejecutada:
SELECT * FROM friends_of_pickles WHERE height_cm < 45 AND species = 'dog';

5

Resultado:



6

Interpretación:
La consulta descarta a Sneakers, que es perro pero mide 55 cm, y a los gatos y humanos, que no cumplen la condición de especie. Quedan solo los perros con altura menor a 45 cm.

7

Aprendizaje:
Con AND cada condición se evalúa por separado y ambas deben cumplirse. Es importante que cada condición tenga su propia referencia completa, como height_cm < 45 AND species = 'dog', porque no funciona igual que en el lenguaje natural.

8

Aplicación en el mundo real:
En seguridad, AND permite combinar filtros, por ejemplo buscar cuentas de administrador (rol = 'admin') que estén inactivas (activa = 0), o accesos fuera de horario (hora < 7) hechos desde un equipo no autorizado, para priorizar revisiones de auditoría.

1

Ejercicio 7: OR (buscar filas que cumplan al menos una condición)

2

Objetivo del ejercicio:
Obtener los amigos de Pickles que son perros **o** miden menos de 45 cm en la tabla friends_of_pickles.

3

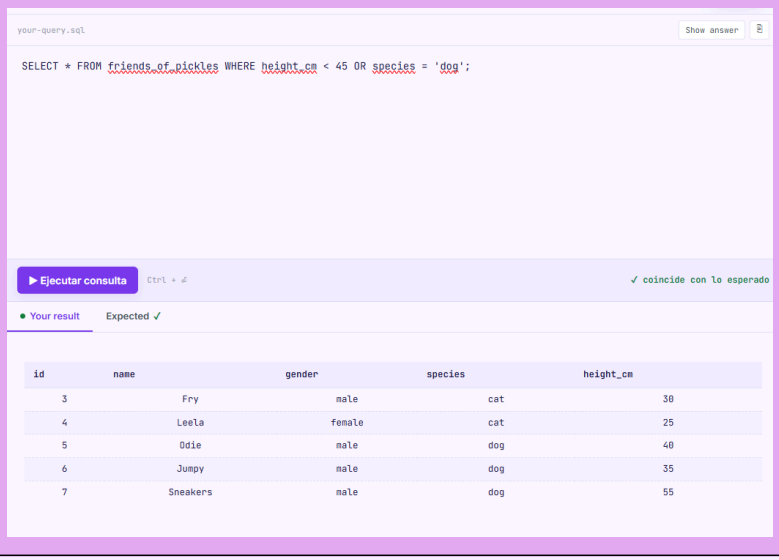
Concepto SQL:
OR: une dos o más condiciones en el WHERE, y una fila se devuelve si cumple al menos una de ellas.

4

Consulta ejecutada:
SELECT * FROM friends_of_pickles WHERE height_cm < 45 OR species = 'dog';

5

Resultado:



6

Interpretación:

La consulta incluye a todos los perros, aunque Sneakers mida 55 cm, y a todos los que miden menos de 45 cm, aunque no sean perros, como los gatos Fry y Leela. Dave (humano, 180 cm) y Mary (humana, 160 cm) quedan fuera porque no cumplen ninguna de las dos condiciones.

7

Aprendizaje:

Comparado con el ejercicio 6, que usaba AND y devolvía dos filas, OR es más inclusivo y devuelve cinco. Además, AND tiene prioridad sobre OR al evaluarse, así que si combinas ambos operadores en una misma consulta conviene usar paréntesis para dejar clara la lógica, por ejemplo WHERE (a OR b) AND c.

8

Aplicación en el mundo real:

En seguridad, OR sirve para generar alertas amplias: por ejemplo, seleccionar registros con más de 5 intentos fallidos de inicio de sesión **o** accesos desde una IP no autorizada **o** accesos fuera de horario, para revisarlos en una auditoría.

1

Ejercicio 8: IN (buscar valores dentro de una lista)

2

Objetivo del ejercicio:

Obtener los amigos de Pickles que **no** son gatos ni perros en la tabla friends_of_pickles.

3

Concepto SQL:

IN permite comparar una columna contra una lista de valores. NOT IN hace lo contrario: devuelve las filas cuyo valor no está en la lista.

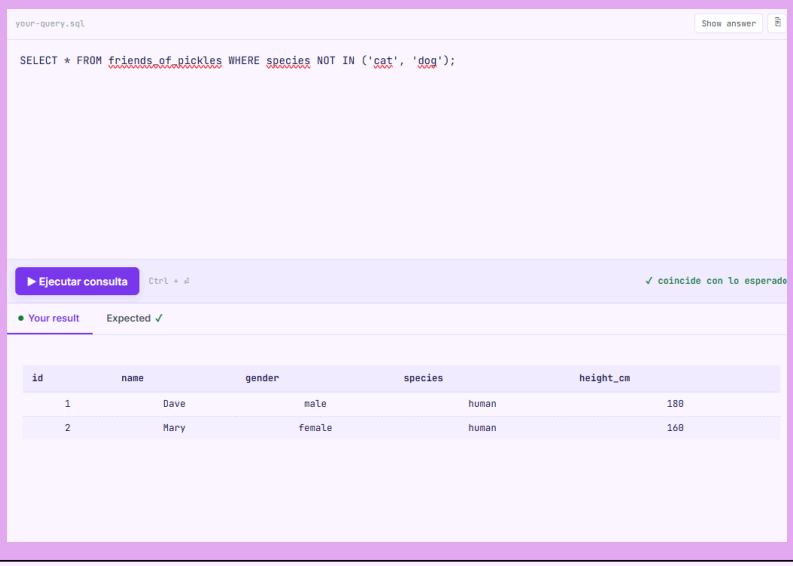
4

Consulta ejecutada:

SELECT * FROM friends_of_pickles WHERE species NOT IN ('cat', 'dog');

5

Resultado:



6

Interpretación:

La consulta descarta a todos los gatos (Fry y Leela) y a todos los perros (Odie, Jumpy y Sneakers), y deja solo a los humanos, que son los únicos amigos de Pickles que no son gatos ni perros.

7

Aprendizaje:
IN es más corto y legible que encadenar varios OR con el mismo campo. Por ejemplo, species IN ('cat', 'dog') equivale a species = 'cat' OR species = 'dog'. Y NOT IN es la versión negada.

8

Aplicación en el mundo real:
En seguridad, IN sirve para filtrar por listas de valores, como usuarios con roles específicos (rol IN ('admin', 'soporte')) o equipos de ciertas áreas. NOT IN permite excluir listas conocidas, por ejemplo descartar cuentas de servicio o dispositivos ya aprobados para revisar solo los que no están en la lista blanca.

1

Ejercicio 9: DISTINCT (eliminar filas duplicadas)

2

Objetivo del ejercicio:
Obtener la lista de especies distintas de los amigos de Pickles que miden más de 50 cm en friends_of_pickles.

3

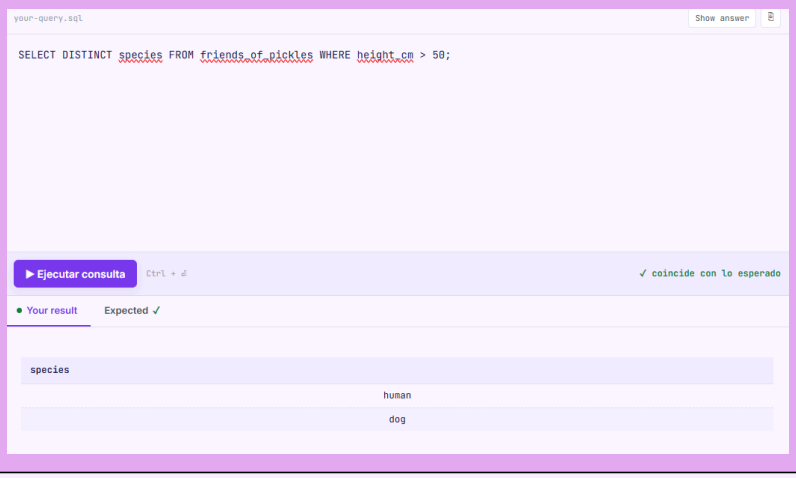
Concepto SQL:
DISTINCT: se coloca después de SELECT y elimina los valores repetidos del resultado

4

Consulta ejecutada:
SELECT DISTINCT species FROM friends_of_pickles WHERE height_cm > 50;

5

Resultado:



6

Interpretación:
Con WHERE height_cm > 50 se conservan Dave (human, 180 cm), Mary (human, 160 cm) y Sneakers (dog, 55 cm). Como DISTINCT elimina repeticiones, las dos humanas aparecen una sola vez, igual que el perro, y el resultado muestra solo las especies: human y dog.

7

Aprendizaje:
DISTINCT no reduce las filas por sí solo, sino que elimina duplicados entre las columnas que se seleccionan. Si pides SELECT DISTINCT gender, species, la combinación cuenta como una sola fila aunque haya varios registros iguales. Además, DISTINCT se aplica después del WHERE, así que primero se filtra y luego se eliminan repetidos.

8

Aplicación en el mundo real:
DISTINCT sirve para saber qué valores existen sin ver cada registro, por ejemplo: qué direcciones IP distintas accedieron a un sistema en una semana, qué roles de usuario están asignados en una aplicación, o qué versiones de software están instaladas en los equipos de una red, para revisar cuáles están desactualizadas.

1

Ejercicio 10: ORDER BY (ordenar filas)

2

Objetivo del ejercicio:
Ordenar los registros de friends_of_pickles por la altura (height_cm) de mayor a menor.

3

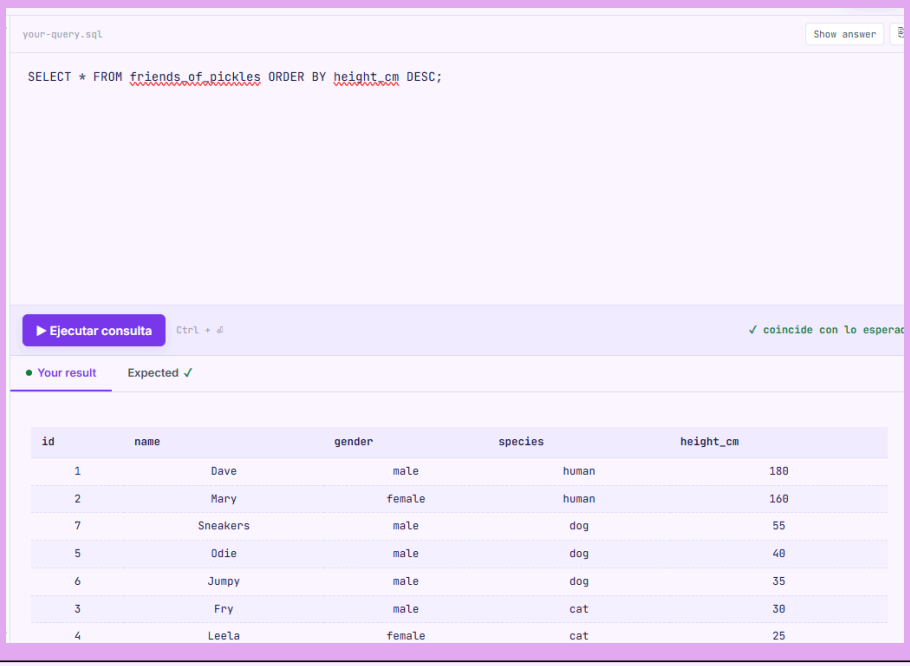
Concepto SQL:
ORDER BY: ordena el resultado por una o más columnas. Por defecto ordena de forma ascendente; con DESC lo hace de forma descendente.

4

Consulta ejecutada:
SELECT * FROM friends_of_pickles ORDER BY height_cm DESC;

5

Resultado:



6

Interpretación:
La consulta devuelve todos los amigos de Pickles, pero ya no en el orden de id. El más alto aparece primero y el más bajo al final.

7

Aprendizaje:
ORDER BY va después del FROM (y del WHERE, si lo hay). Sin DESC el orden es ascendente, así que conviene escribirlo explícitamente cuando importe la dirección. Además, la base de datos no garantiza ningún orden si no usas ORDER BY, aunque a veces parezca ordenado.

8

Aplicación en el mundo real:
En seguridad, ORDER BY sirve para priorizar revisiones, por ejemplo ordenar las cuentas por número de intentos fallidos de forma descendente para atender primero las más sospechosas, o ordenar los eventos de un log por fecha para reconstruir la secuencia de un incidente.

1

Ejercicio 11: LIMIT (limitar el número de filas devueltas)

2

Objetivo del ejercicio:
Obtener solo la fila del amigo más alto de Pickles en friends_of_pickles.

3

Concepto SQL:
LIMIT: restringe el número de filas que devuelve la consulta. Suele usarse junto con ORDER BY para elegir las primeras filas de un orden determinado.

4

Consulta ejecutada:
SELECT * FROM friends_of_pickles ORDER BY height_cm DESC LIMIT 1;

5

Resultado:



6

Interpretación:
La consulta ordena a los amigos de mayor a menor altura y se queda solo con el primero, que es Dave con 180 cm.

7

Aprendizaje:
LIMIT se coloca al final de la consulta. Si no se combina con ORDER BY, devuelve filas sin un orden definido, así que para obtener "las primeras" o "las más altas" es necesario ordenar primero.

8

Aplicación en el mundo real:
En seguridad, LIMIT permite revisar una muestra de registros de una tabla muy grande, por ejemplo, las 10 entradas más recientes de un log de accesos, o los 5 equipos con más alertas para priorizar su revisión.

1

Ejercicio 12: COUNT(*) (contar filas)

2

Objetivo del ejercicio:
Saber cuántas filas tiene la tabla friends_of_pickles sin tener que verlas todas.

3

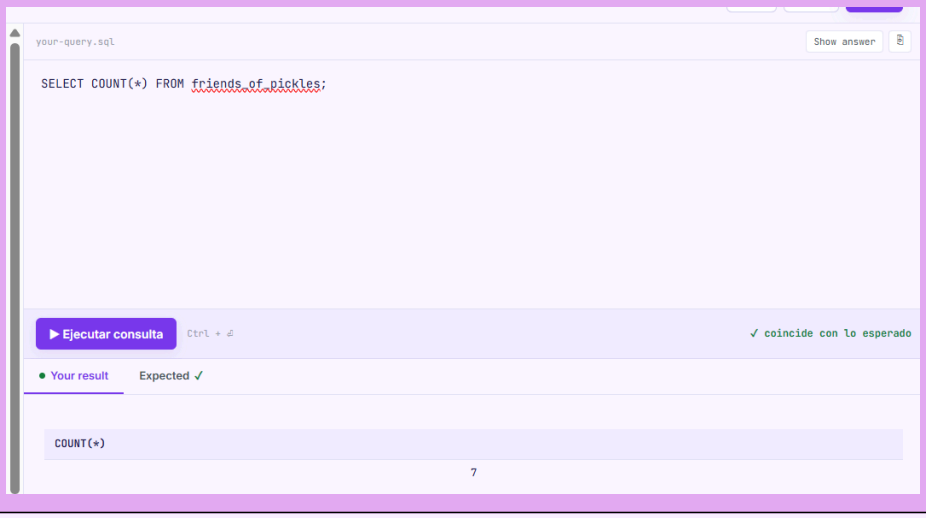
Concepto SQL:
COUNT(*): es una función de agregación que cuenta el número total de filas que devuelve la consulta.

4

Consulta ejecutada:
SELECT COUNT(*) FROM friends_of_pickles;

5

Resultado:



6

Interpretación:
La tabla tiene siete registros, que corresponden a los siete amigos de Pickles: Dave, Mary, Fry, Leela, Odie, Jumpy y Sneakers

7

Aprendizaje:
A diferencia de SELECT *, que devuelve las filas, COUNT(*) devuelve un único número. Las funciones de agregación resumen varias filas en un valor, y más adelante se combinarán con GROUP BY y WHERE.

8

Aplicación en el mundo real:
En seguridad, COUNT(*) sirve para dimensionar un problema antes de investigarlo, por ejemplo cuántos intentos fallidos de acceso hubo en un día, cuántos equipos no tienen parche de seguridad o cuántos registros de una tabla de usuarios tienen privilegios de administrador.

1

Ejercicio 13: COUNT(*) ... WHERE (contar filas que cumplen una condición)

2

Objetivo del ejercicio:
Saber cuántos amigos de Pickles son perros en la tabla friends_of_pickles.

3

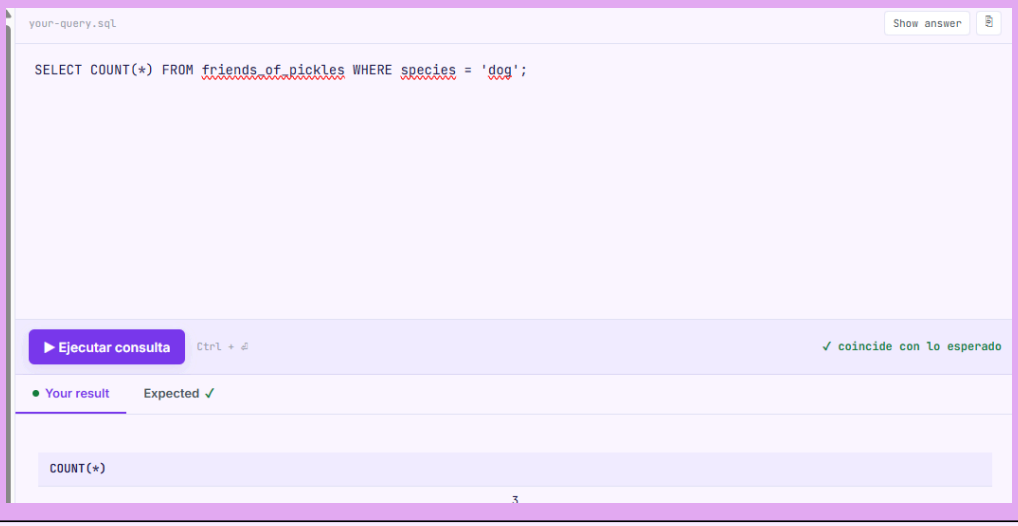
Concepto SQL:
COUNT(*) combinado con WHERE: primero se filtran las filas que cumplen la condición y después se cuentan.

4

Consulta ejecutada:
SELECT COUNT(*) FROM friends_of_pickles WHERE species = 'dog';

5

Resultado:



6

Interpretación:
Hay tres perros entre los amigos de Pickles: Odie, Jumpy y Sneakers. Los gatos y humanos no se cuentan porque no cumplen la condición.

7

Aprendizaje:
El WHERE actúa antes del conteo, así que COUNT(*) solo cuenta las filas filtradas. Es el mismo filtro del ejercicio 3, pero ahora en lugar de mostrar las filas, se devuelve su cantidad.

8

Aplicación en el mundo real:
En seguridad, esta combinación permite cuantificar hallazgos, por ejemplo cuántas cuentas tienen privilegios de administrador sin MFA activado, cuántos intentos fallidos hubo desde una IP específica, o cuántos equipos de un área no cumplen la política de parches, para reportarlo en una auditoría.

1

Ejercicio 14: SUM (sumar valores de una columna)

2

Objetivo del ejercicio:
Calcular el total de libros leídos por la familia en la tabla family_members.

3

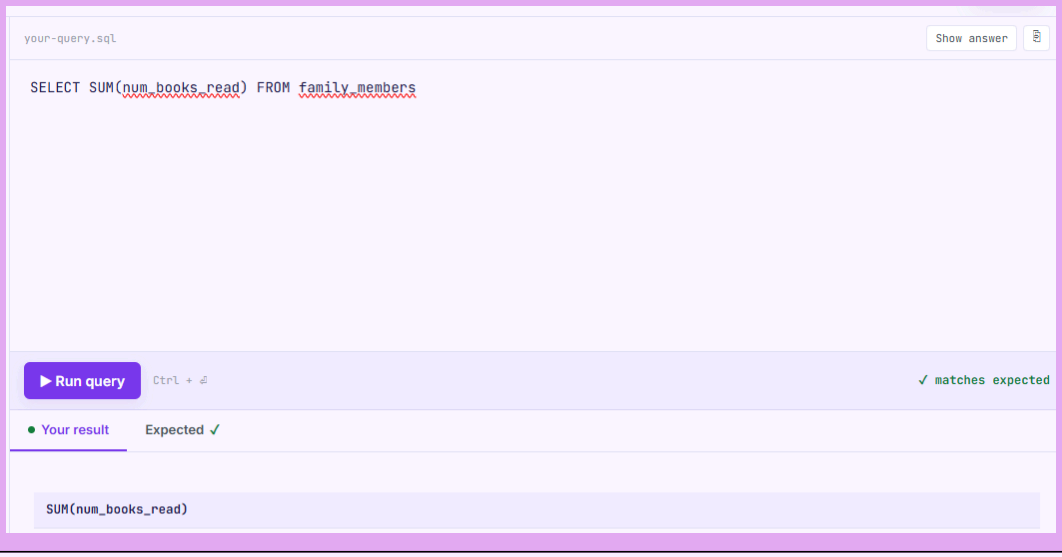
Concepto SQL:
SUM(columna): es una función de agregación que suma todos los valores numéricos de una columna.

4

Consulta ejecutada:
SELECT SUM(num_books_read) FROM family_members;

5

Resultado:



6

Interpretación:
Dave leyó 200 libros, Mary 180 y Pickles 0. La suma de esos valores es 380, que es el total de libros leídos por la familia

7

Aprendizaje:
SUM resume varias filas en un solo número, igual que COUNT(*), pero suma valores en lugar de contar filas. Solo funciona con columnas numéricas. Además, la tabla tiene una columna num_legs que no usé, pero funciona de la misma forma si la necesitas.

8 **Aplicación en el mundo real:**
En seguridad, SUM sirve para totalizar métricas, por ejemplo la cantidad total de megabytes transferidos desde un servidor en una semana para detectar fugas de datos, el total de incidentes registrados por área, o el número total de intentos fallidos acumulados por una cuenta.

---

1 **Ejercicio 15: AVG (calcular el promedio de una columna)**

2 **Objetivo del ejercicio:**
Calcular el promedio de libros leídos por miembro de la familia en la tabla family_members.

3 **Concepto SQL:**
AVG(columna): es una función de agregación que calcula el promedio de los valores numéricos de una columna.

4 **Consulta ejecutada:**
SELECT AVG(num_books_read) FROM family_members;

5 **Resultado:**



6 **Interpretación:**
La suma de libros es 380 y hay tres miembros, así que el promedio es 380 / 3 ≈ 126.67. Pickles, con 0 libros, baja el promedio, aunque Dave y Mary leen bastante más.

7 **Aprendizaje:**
AVG calcula la suma dividida entre el número de filas con valor. Como la computadora maneja los decimales de forma aproximada, el resultado puede mostrar muchos decimales. Si se necesita un número redondeado, se puede usar ROUND().

8 **Aplicación en el mundo real:**
En seguridad, AVG sirve para establecer líneas base de comportamiento, por ejemplo el promedio de intentos de inicio de sesión por usuario al día, o el tiempo promedio de respuesta a incidentes. Un valor muy alejado del promedio puede señalar una anomalía que conviene revisar.

---

1 **Ejercicio 16: MAX y MIN (valor máximo y mínimo de una columna)**

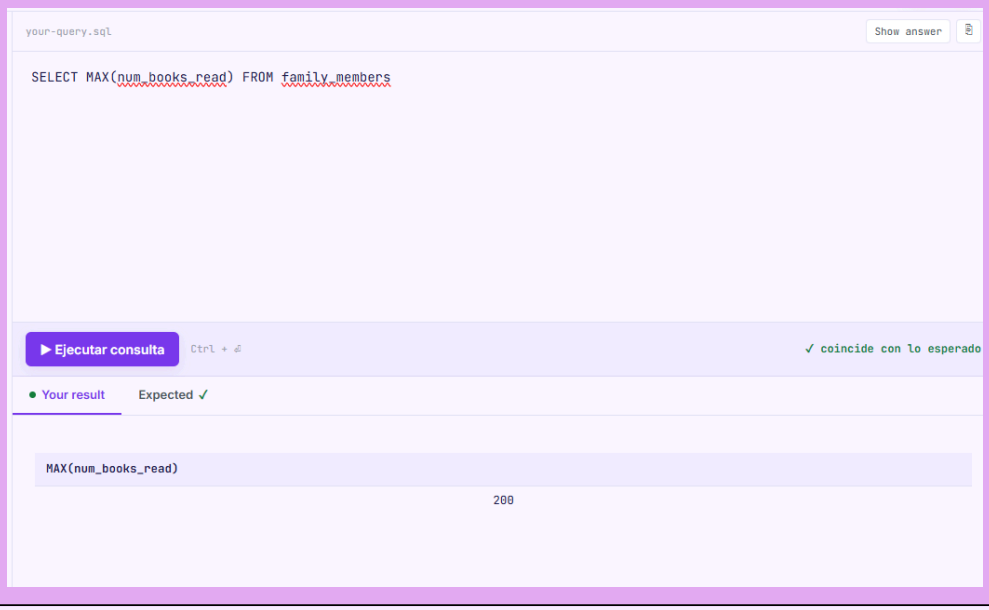
2 **Objetivo del ejercicio:**
Encontrar el número más alto de libros leídos por un miembro de la familia en la tabla family_members.

3 **Concepto SQL:**
MAX(columna) devuelve el valor más alto de una columna y MIN(columna) el más bajo. Son funciones de agregación, como SUM y AVG.

4 **Consulta ejecutada:**
SELECT MAX(num_books_read) FROM family_members;

5

Resultado:



6

Interpretación:
De los tres miembros, Dave leyó la mayor cantidad de libros, con 200. Mary tiene 180 y Pickles 0.

7

Aprendizaje:
MAX y MIN no solo funcionan con números: también pueden aplicarse a texto o fechas, devolviendo el valor más alto o más bajo según el orden de esa columna. Además, pueden combinarse con WHERE, como los demás agregados.

8

Aplicación en el mundo real:
En seguridad, MAX puede usarse para encontrar la fecha del último acceso de una cuenta o el último parche aplicado a un equipo, y MIN para la primera fecha de un evento sospechoso, lo que ayuda a delimitar la ventana de un incidente.

1

Ejercicio 17: GROUP BY (agrupar filas por una columna)

2

Objetivo del ejercicio:
Obtener la altura máxima de cada especie en la tabla friends_of_pickles, mostrando el nombre de la especie junto a su altura.

3

Concepto SQL:
GROUP BY: divide las filas en grupos según el valor de una columna, y las funciones de agregación (MAX, COUNT, SUM, etc.) se aplican a cada grupo por separado.

4

Consulta ejecutada:
SELECT MAX(height_cm), species FROM friends_of_pickles GROUP BY species;

5

Resultado:



6 **Interpretación:**
La consulta separa a los amigos de Pickles en tres grupos por especie. En cada grupo toma la altura más alta: entre los gatos Fry (30) y Leela (25) gana Fry; entre los perros, Sneakers (55); y entre los humanos, Dave (180).

7 **Aprendizaje:**
Cada columna que no es una función de agregación debe aparecer en el GROUP BY. Además, el resultado no sigue un orden fijo si no usas ORDER BY, aunque aquí salió cat, dog, human.

8 **Aplicación en el mundo real:**
En seguridad, GROUP BY permite resumir por categoría, por ejemplo contar intentos fallidos por usuario, alertas por tipo de incidente o equipos sin parche por área, para identificar dónde se concentran los problemas.

---

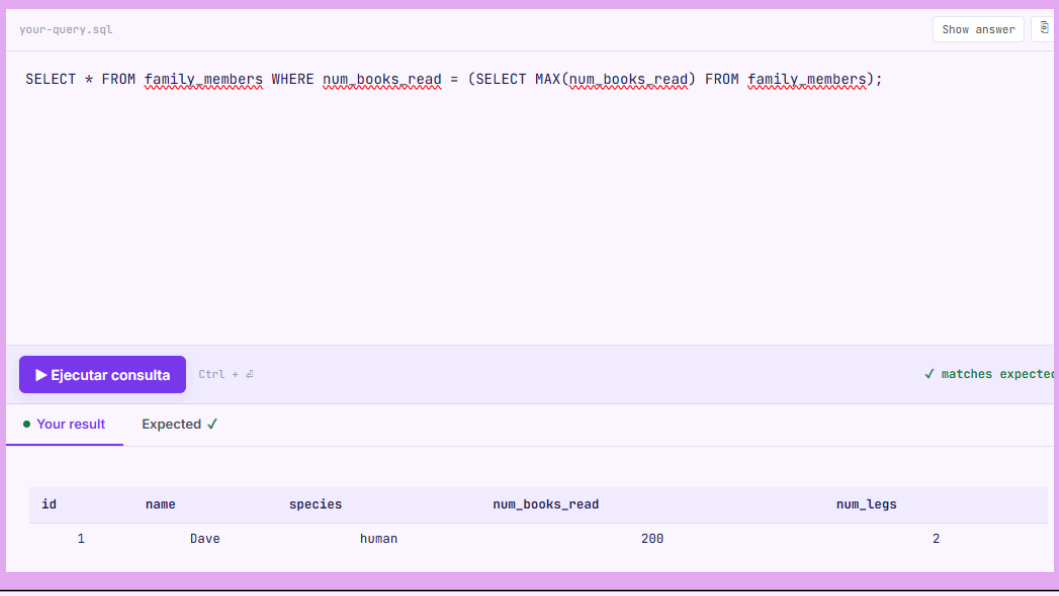
1 **Ejercicio 18: Nested queries (consultas anidadas)**

2 **Objetivo del ejercicio:**
Obtener los miembros de family_members que tienen el mayor número de libros leídos.

3 **Concepto SQL:**
Subconsulta: una consulta dentro de otra, entre paréntesis. La interna se ejecuta primero y su resultado se usa en la externa.

4 **Consulta ejecutada:**
SELECT * FROM family_members WHERE num_books_read = (SELECT MAX(num_books_read) FROM family_members);

5 **Resultado:**



6 **Interpretación:**
La subconsulta calcula primero el máximo de libros leídos, que es 200. La consulta externa usa ese valor para devolver solo a los miembros que lo igualan, en este caso Dave.

7 **Aprendizaje:**
Una subconsulta puede reemplazar un valor fijo, lo que evita escribir números a mano y hace que la consulta se actualice sola si cambian los datos. Si la subconsulta devuelve más de un valor, con = daría error; en ese caso se usa IN.

8 **Aplicación en el mundo real:**
En seguridad, las subconsultas permiten comparar contra un valor calculado, por ejemplo buscar los usuarios cuyo número de accesos supera el promedio de la organización, o los equipos cuya versión de software es anterior a la más reciente aprobada en la política.

---

1 **Ejercicio 19: NULL (valores nulos, filas sin valor en una columna)**

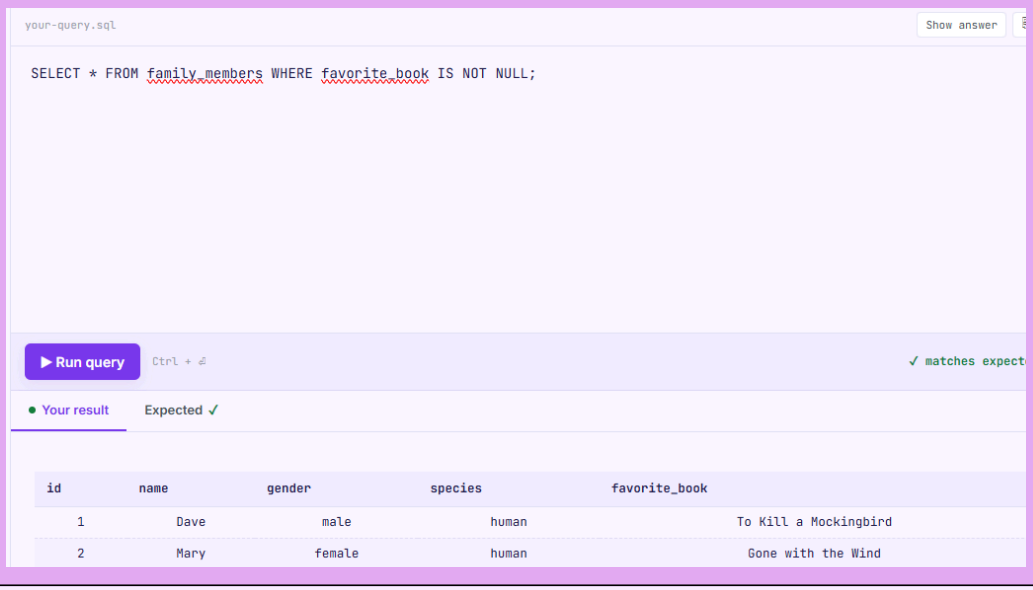
2 **Objetivo del ejercicio:**
Obtener los miembros de family_members que sí tienen un libro favorito registrado, es decir, cuyo favorite_book no es nulo.

3 **Concepto SQL:**
NULL representa la ausencia de valor en una columna. Para filtrarlo se usa IS NULL o IS NOT NULL, nunca = NULL.

4 **Consulta ejecutada:**
SELECT * FROM family_members WHERE favorite_book IS NOT NULL;

5

Resultado:



6

Interpretación:
La consulta descarta a Pickles, que es un perro y no tiene libro favorito, así que su columna favorite_book está vacía (NULL). Dave y Mary sí tienen uno registrado.

7

Aprendizaje:
NULL no es lo mismo que cero ni que texto vacío: significa que no hay dato. Por eso no se compara con =, sino con IS NULL o IS NOT NULL. Además, un NULL no cumple condiciones como favorite_book = 'x' ni favorite_book <> 'x', así que puede quedar fuera de ambos resultados sin aviso.

8

Aplicación en el mundo real:
En seguridad, IS NULL ayuda a detectar datos faltantes que pueden ser un riesgo, por ejemplo cuentas sin correo de recuperación o sin fecha de último cambio de contraseña, equipos sin registro de parche aplicado o usuarios sin área asignada.

1

Ejercicio 20: Date (comparar fechas)

2

Objetivo del ejercicio:
Obtener los famosos de la tabla celebs_born que nacieron después del 1 de septiembre de 1980.

3

Concepto SQL:
Comparación de fechas con > y <. Las fechas se escriben como texto entre comillas en formato 'AAAA-MM-DD', y se comparan en orden cronológico.

4

Consulta ejecutada:
SELECT * FROM celebs_born WHERE birthdate > '1980-09-1';

5

Resultado:



6

Interpretación:
La consulta descarta a Michael Jordan, nacido en 1963, y devuelve solo a las dos celebridades nacidas después de esa fecha.

7

Aprendizaje:
Las fechas se comparan como si fueran números, por eso el formato AAAA-MM-DD debe ser siempre el mismo. Además, la fecha de la consulta debería escribirse con dos dígitos en el día y el mes ('1980-09-01'), que es el formato estándar; en este caso el sistema la aceptó, pero en otros motores podría dar un resultado distinto o un error.

8

Aplicación en el mundo real:
En seguridad, las comparaciones de fechas permiten filtrar por periodos, por ejemplo accesos posteriores a la fecha de un incidente, cuentas creadas después de una fecha de cambio de política, o certificados que vencen antes de una fecha límite.

1

Ejercicio 21: Inner joins (unir tablas con INNER JOIN)

2

Objetivo del ejercicio:
Obtener el nombre de cada personaje junto con el nombre del actor que lo interpreta, combinando las tablas character y character_actor.

3

Concepto SQL:
INNER JOIN ... ON: combina filas de dos tablas cuando el valor de una columna coincide con el de la otra. Por ejemplo, character.id = character_actor.character_id. Se usa la notación tabla.columna para indicar de qué tabla viene cada columna.

4

Consulta ejecutada:
SELECT character.name, character_actor.actor_name FROM character INNER JOIN character_actor ON character.id = character_actor.character_id;

5

Resultado:

```
SELECT character.name, character_actor.actor_name
FROM character
INNER JOIN character_actor
ON character.id = character_actor.character_id;
```

▶ Run query Ctrl + ⏎ ✓ matches expecte

● Your result Expected ✓

name	actor_name
Doogie Howser	Neil Patrick Harris
Barney Stinson	Neil Patrick Harris
Lily Aldrin	Alyson Hannigan
Willow Rosenberg	Alyson Hannigan

6

Interpretación:
La consulta toma cada fila de character y la empareja con la fila de character_actor que tiene el mismo character_id. Así se obtiene el personaje junto con su actor, sin repetir los nombres en las tablas.

7

Aprendizaje:
Los datos se guardan separados para no duplicarlos: si cambia el nombre de un personaje, solo se actualiza una fila. El JOIN los reúne al momento de consultar. Además, al unir tablas conviene usar tabla.columna para evitar ambigüedades cuando ambas tablas tienen columnas con el mismo nombre.

8

Aplicación en el mundo real:
En seguridad, los INNER JOIN permiten cruzar información de distintas fuentes, por ejemplo unir la tabla de usuarios con la de accesos para ver quién ingresó a qué sistema, o la de equipos con la de parches para saber qué máquinas tienen una actualización pendiente.

1

Ejercicio 22: Multiple joins (unir más de dos tablas)

2

Objetivo del ejercicio:
Obtener el nombre de cada personaje junto con el nombre del actor que lo interpreta, usando dos JOIN para pasar por la tabla de relación character_actor y llegar a la tabla actor.

3

Concepto SQL:
Varios INNER JOIN encadenados: cada JOIN une una tabla nueva con una de las ya unidas, usando la columna que tienen en común.

4

Consulta ejecutada:
SELECT character.name, actor.name FROM character INNER JOIN character_actor ON character.id = character_actor.character_id INNER JOIN actor ON character_actor.actor_id = actor.id;

5

Resultado:

your-query.sql — Show answer

```
SELECT character.name, actor.name
FROM character
INNER JOIN character_actor
ON character.id = character_actor.character_id
INNER JOIN actor
ON character_actor.actor_id = actor.id;
```

▶ Run query Ctrl + ⏎ ✓ matches expected

● Your result Expected ✓

name	name
Doogie Howser	Neil Patrick Harris
Barney Stinson	Neil Patrick Harris
Lily Aldrin	Alyson Hannigan
Willow Rosenberg	Alyson Hannigan

6

Interpretación:
La consulta primero une cada personaje con su registro en character_actor y luego usa el actor_id de ese registro para buscar el nombre en actor. El resultado es igual al del ejercicio 21, pero ahora el nombre del actor sale de su propia tabla, sin estar duplicado.

7

Aprendizaje:
Cada JOIN necesita una condición ON que relacione la tabla nueva con una ya presente. Como ambas columnas se llaman name, en el resultado aparecen con el mismo encabezado, así que conviene usar alias (por ejemplo, actor.name AS actor_name) para distinguirlas.

8

Aplicación en el mundo real:
En seguridad, unir varias tablas permite reconstruir una cadena completa, por ejemplo: usuario → equipo asignado → vulnerabilidades del equipo → responsable del área, para saber a quién notificar y qué corregir.

1

Ejercicio 23: Joins with WHERE (unir tablas y filtrar con WHERE)

2

Objetivo del ejercicio:
Obtener los personajes y las series en las que aparecen, excluyendo a Willow Rosenberg y las filas de la serie How I Met Your Mother, en la tabla character, character_tv_show y tv_show.

3

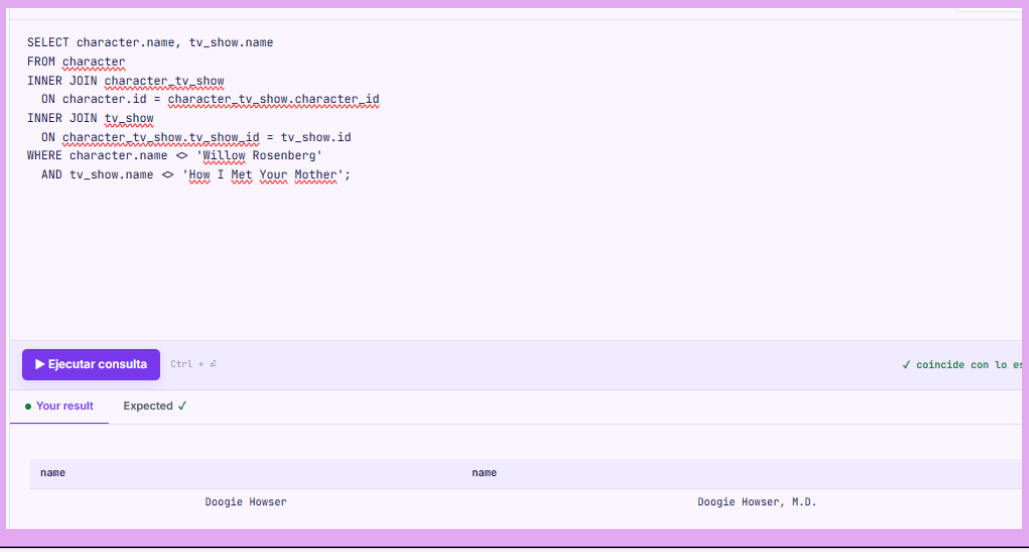
Concepto SQL:
Varios INNER JOIN encadenados para unir las tres tablas, seguidos de un WHERE con dos condiciones unidas por AND. El operador <> significa "distinto de".

4

Consulta ejecutada:
SELECT character.name, tv_show.name FROM character INNER JOIN character_tv_show ON character.id = character_tv_show.character_id INNER JOIN tv_show ON character_tv_show.tv_show_id = tv_show.id WHERE character.name <> 'Willow Rosenberg' AND tv_show.name <> 'How I Met Your Mother';

5

Resultado:



6

Interpretación:

Los INNER JOIN relacionan cada personaje con su serie. El primer filtro descarta a Willow Rosenberg, que aparece en Buffy the Vampire Slayer. El segundo descarta a Barney Stinson y Lily Aldrin, que están en How I Met Your Mother. Solo queda Doogie Howser, que está en Doogie Howser, M.D.

7

Aprendizaje:

Con AND, una fila debe cumplir las dos condiciones para aparecer. Al usar <> en el segundo filtro, la lógica cambia por completo: en lugar de quedarme con una serie, la excluyo. Mi primera versión usaba = y devolvía a Barney y Lily, así que conviene leer con cuidado si la condición pide "distinto de" o "igual a".

8

Aplicación en el mundo real:

En seguridad, este tipo de filtro permite excluir resultados conocidos para revisar solo lo nuevo, por ejemplo listar los equipos de una red excluyendo los que ya están en la lista de equipos aprobados, o las cuentas de usuario excluyendo las cuentas de servicio y las ya revisadas en la auditoría anterior.

1

Ejercicio 24: Left joins (unir tablas conservando todas las filas de la tabla de la izquierda)

2

Objetivo del ejercicio:

Obtener todos los nombres de personajes con el nombre de su actor, incluidos los personajes que no tienen actor registrado.

3

Concepto SQL:

LEFT JOIN ... ON: devuelve todas las filas de la tabla de la izquierda y, si no hay coincidencia en la tabla de la derecha, rellena esas columnas con NULL.

4

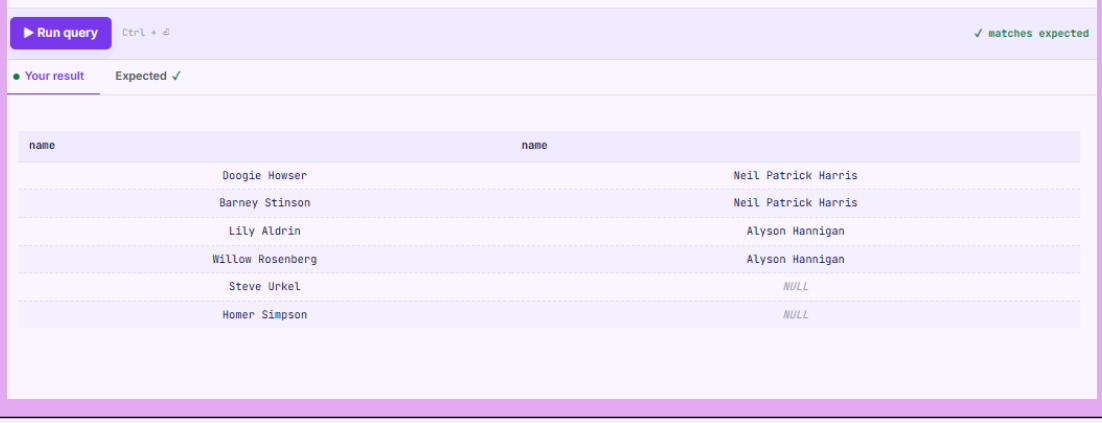
Consulta ejecutada:

SELECT character.name, actor.name FROM character LEFT JOIN character_actor ON character.id = character_actor.character_id LEFT JOIN actor ON character_actor.actor_id = actor.id;

5

Resultado:

```
SELECT character.name, actor.name FROM character LEFT JOIN character_actor ON character.id = character_actor.character_id LEFT JOIN actor ON
character_actor.actor_id = actor.id;
```



name	name
Doogie Howser	Neil Patrick Harris
Barney Stinson	Neil Patrick Harris
Lily Aldrin	Alyson Hannigan
Willow Rosenberg	Alyson Hannigan
Steve Urkel	NULL
Homer Simpson	NULL

6

Interpretación:

Los primeros cuatro personajes tienen actor en character_actor, así que se emparejan como en el ejercicio 22. Steve Urkel y Homer Simpson no tienen registro en character_actor, así que el segundo LEFT JOIN no encuentra actor para ellos y su columna aparece vacía (NULL). Con INNER JOIN estos dos personajes habrían desaparecido del resultado.

7

Aprendizaje:

La diferencia clave entre INNER JOIN y LEFT JOIN está en qué filas se conservan: el INNER solo deja las coincidencias, mientras que el LEFT conserva todas las de la tabla de la izquierda. Además, el orden de las tablas importa, porque la tabla que va primero en el FROM es la que se conserva completa. SQLite no tiene RIGHT JOIN ni OUTER JOIN, como se indica en el enunciado.

8

Aplicación en el mundo real:

En seguridad, un LEFT JOIN sirve para encontrar lo que falta. Por ejemplo, listar todas las cuentas de usuario y unirlas con el registro de MFA: las cuentas con NULL en esa columna no tienen MFA activado y deben revisarse. Lo mismo aplica a equipos sin escaneo de vulnerabilidades o activos sin responsable asignado.

1

Ejercicio 25: Table alias (alias de tablas con AS)

2

Objetivo del ejercicio:

Obtener los nombres de los personajes con los nombres de sus actores, igual que en el ejercicio 24, pero usando alias más cortos para cada tabla.

3

Concepto SQL:

Alias de tabla con AS: asigna un nombre corto a una tabla para referirse a sus columnas sin escribir el nombre completo. En este caso, character AS c, character_actor AS ca y actor AS a..

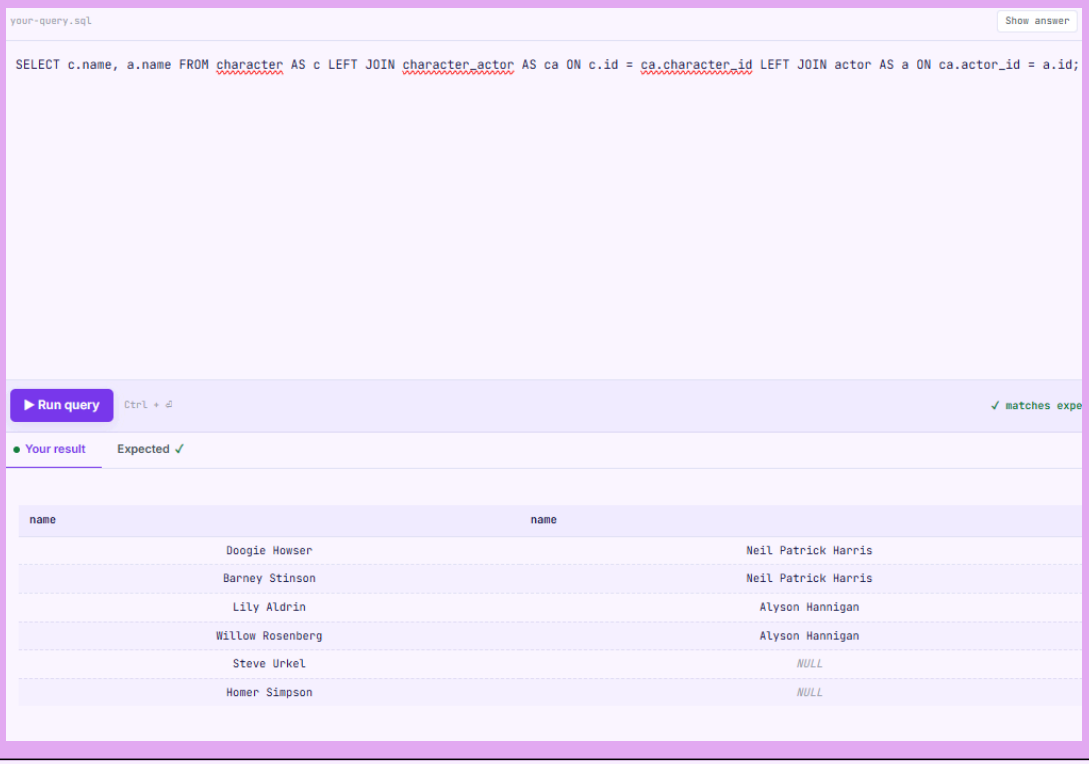
4

Consulta ejecutada:

SELECT c.name, a.name FROM character AS c LEFT JOIN character_actor AS ca ON c.id = ca.character_id LEFT JOIN actor AS a ON ca.actor_id = a.id;

5

Resultado:



6

Interpretación:
La consulta devuelve exactamente lo mismo que el ejercicio 24. Los alias solo cambian cómo se escribe la consulta, no el resultado. Steve Urkel y Homer Simpson siguen apareciendo con NULL porque el LEFT JOIN conserva todos los personajes.

7

Aprendizaje:
Con alias, las condiciones ON y las columnas del SELECT se vuelven más cortas y fáciles de leer. Una vez definido el alias en el FROM o en un JOIN, hay que usarlo en el resto de la consulta, no el nombre original de la tabla. El AS es opcional en muchos motores, pero escribirlo hace más clara la consulta.

8

Aplicación en el mundo real:
En seguridad, las consultas que cruzan muchas tablas, como usuarios, roles, permisos y accesos, se vuelven largas rápidamente. Los alias permiten mantenerlas legibles para una auditoría o para compartirlas con el equipo, sin cambiar el resultado.

1

Ejercicio 26: Column alias (alias de columnas con AS)

2

Objetivo del ejercicio:
Obtener los nombres de los personajes y de sus actores con LEFT JOIN, y nombrar las columnas del resultado como character y actor.

3

Concepto SQL:
Alias de columna con AS: cambia el encabezado de una columna en el resultado sin modificar los datos. Por ejemplo, c.name AS character.

4

Consulta ejecutada:
SELECT c.name AS character, a.name AS actor FROM character AS c LEFT JOIN character_actor AS ca ON c.id = ca.character_id LEFT JOIN actor AS a ON ca.actor_id = a.id;

5

Resultado:

```
SELECT c.name AS character, a.name AS actor FROM character AS c LEFT JOIN character_actor AS ca ON c.id = ca.character_id LEFT JOIN actor AS a ON ca.actor_id = a.id;
```

▶ Run query Ctrl + ⏎ ✓ matches expected

● Your result Expected ✓

character	actor
Doogie Howser	Neil Patrick Harris
Barney Stinson	Neil Patrick Harris
Lily Aldrin	Alyson Hannigan
Willow Rosenberg	Alyson Hannigan
Steve Urkel	NULL
Homer Simpson	NULL

6

Interpretación:

Los datos son los mismos que en los ejercicios 24 y 25. Lo que cambia son los encabezados, que ahora dicen character y actor en lugar de name repetido. Steve Urkel y Homer Simpson aparecen con NULL porque el LEFT JOIN conserva a todos los personajes, aunque no tengan actor registrado.

7

Aprendizaje:

Un alias de columna solo afecta cómo se muestra el resultado, no la forma en que se calcula. Además, en el WHERE o en el ORDER BY no siempre se puede usar el alias de columna, según el motor. Por eso conviene distinguir entre alias de tabla (para escribir menos) y alias de columna (para aclarar el resultado).

8

Aplicación en el mundo real:

En seguridad, los alias de columna sirven para que un reporte de auditoría sea claro para quien lo lee, por ejemplo usuario.nombre AS responsable o equipo.nombre AS activo. Así no hay confusión entre columnas que originalmente se llaman igual.

1

Ejercicio 27: Self joins (unir una tabla consigo misma)

2

Objetivo del ejercicio:

Obtener el nombre de cada empleado junto con el nombre de su jefe, usando la tabla employees.

3

Concepto SQL:

Self join: la misma tabla aparece dos veces en el FROM con alias distintos (e1 y e2), de modo que se pueden comparar filas de la tabla con otras filas de esa misma tabla.

4

Consulta ejecutada:

SELECT e1.name AS employee_name, e2.name AS boss_name FROM employees AS e1 INNER JOIN employees AS e2 ON e1.boss_id = e2.id;

5

Resultado:



6

Interpretación:

Cada empleado (e1) se empareja con la fila de su jefe (e2), cuyo id coincide con el boss_id del empleado. Maxine Tang, la CEO, tiene boss_id NULL, así que no tiene jefe y con INNER JOIN no aparece en el resultado.

7

Aprendizaje:

En un self join, los alias son obligatorios, porque sin ellos el motor no sabe qué copia de la tabla usar. Además, el tipo de join cambia el resultado: con INNER JOIN se pierde a quien no tiene jefe, y con LEFT JOIN aparecería con NULL en boss_name. Hay que confirmar en la plataforma cuál de los dos espera el enunciado.

8

Aplicación en el mundo real:

En seguridad, un self join permite reconstruir cadenas jerárquicas, por ejemplo quién aprueba los accesos de quién, o qué cuenta de administrador creó a otra. Así se identifica la línea de autorización para una auditoría de privilegios.

1

Ejercicio 28: LIKE (buscar patrones en texto)

2

Objetivo del ejercicio:

Obtener los robots cuyo nombre contiene “Robot” seguido de un año entre 2000 y 2099, en la tabla robots.

3

Concepto SQL:

LIKE compara texto con un patrón. % representa cero, uno o varios caracteres, y _ representa exactamente un carácter.

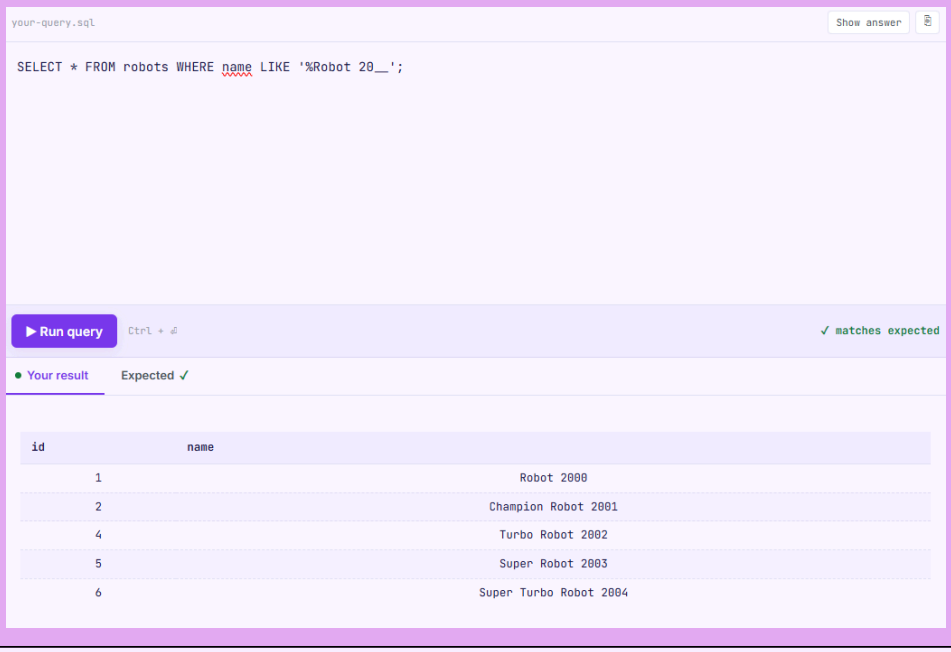
4

Consulta ejecutada:

SELECT * FROM robots WHERE name LIKE '%Robot 20__';

5

Resultado:



6

Interpretación:
% al inicio permite cualquier texto antes de “Robot”. 20__ exige que el nombre termine en “20” seguido de dos caracteres cualesquiera, lo que cubre los años 2000 a 2099. Dragon y Not A Robot no tienen año al final, así que quedan fuera. Unreleased Turbo Robot 2111 también queda fuera, porque 2111 empieza con 21 y no con 20.

7

Aprendizaje:
Cada _ cuenta exactamente un carácter, mientras que % puede contar cero o más. Si el patrón fuera '%Robot%20%', también coincidiría 2111 porque tiene un “20” en medio. Además, LIKE no distingue mayúsculas y minúsculas en SQLite, así que '%robot%' daría el mismo resultado.

8

Aplicación en el mundo real:
En seguridad, LIKE sirve para buscar patrones en nombres de equipos, rutas de archivos o correos, por ejemplo hostname LIKE 'srv-%' para listar servidores, o ruta LIKE '%.exe' para encontrar ejecutables en una carpeta sospechosa.

1

Ejercicio 29: CASE (valores condicionales en una columna calculada)

2

Objetivo del ejercicio:
Crear una columna llamada sound en friends_of_pickles que muestre “talk” para humanos, “bark” para perros y “meow” para gatos.

3

Concepto SQL:
CASE WHEN ... THEN ... ELSE ... END: funciona como un if/else dentro de una consulta. Evalúa las condiciones en orden y devuelve el valor de la primera que se cumple. Con AS se le da nombre a la columna calculada.

4

Consulta ejecutada:
SELECT *, CASE WHEN species = 'human' THEN 'talk' WHEN species = 'dog' THEN 'bark' WHEN species = 'cat' THEN 'meow' END AS sound FROM friends_of_pickles;

5

Resultado:

your-query.sql — Show answer

```
SELECT *, CASE WHEN species = 'human' THEN 'talk' WHEN species = 'dog' THEN 'bark' WHEN species = 'cat' THEN 'meow' END AS sound FROM
friends_of_pickles;
```

▶ Run query Ctrl + ⏎ ✓ matches expected

● Your result Expected ✓

id	name	gender	species	height_cm	sound
1	Dave	male	human	180	talk
2	Mary	female	human	160	talk
3	Fry	male	cat	30	meow
4	Leela	female	cat	25	meow
5	Odie	male	dog	40	bark
6	Jumpy	male	dog	35	bark
7	Sneakers	male	dog	55	bark

6

Interpretación:
La consulta conserva todas las columnas originales con * y agrega sound, que cambia según la especie de cada fila. Los humanos hablan, los perros ladran y los gatos maúllan.

7

Aprendizaje:
CASE permite crear columnas calculadas sin cambiar los datos de la tabla. Si ninguna condición se cumple y no hay ELSE, el resultado es NULL. Por eso conviene agregar un ELSE cuando se espera que siempre haya un valor, por ejemplo ELSE 'desconocido'.

8

Aplicación en el mundo real:
En seguridad, CASE sirve para clasificar hallazgos sin modificar la fuente, por ejemplo marcar una vulnerabilidad como “crítica” si su puntaje CVSS es 9 o más, “alta” si está entre 7 y 9, y “media” en otro caso, para priorizar la remediación.

1

Ejercicio 30: SUBSTR (extraer parte de un texto)

2

Objetivo del ejercicio:
Obtener todos los robots ubicados en NY, usando las dos últimas letras de la columna location.

3

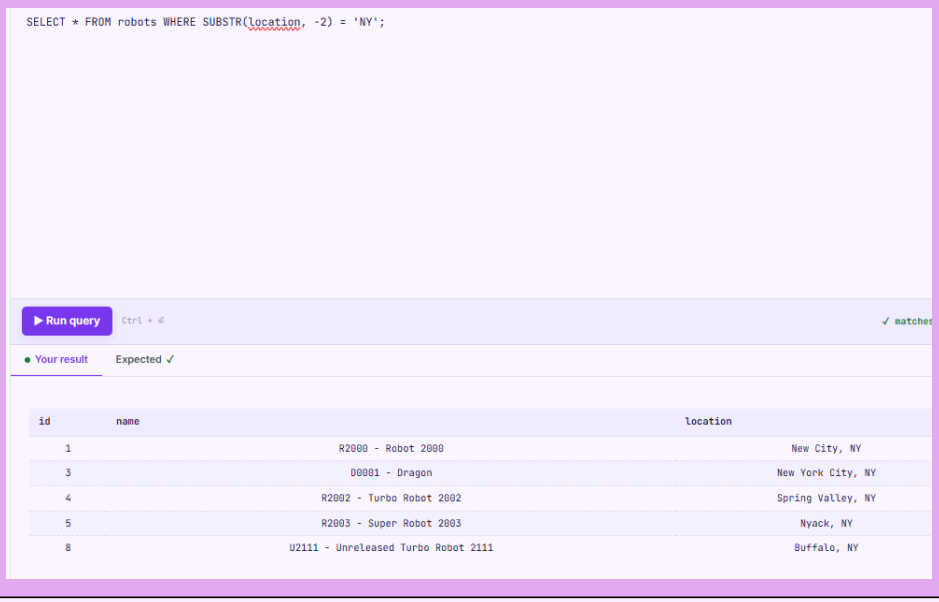
Concepto SQL:
SUBSTR(columna, índice, número_de_caracteres): extrae una parte de un texto. Con un índice negativo cuenta desde el final, así que SUBSTR(location, -2) devuelve las dos últimas letras.

4

Consulta ejecutada:
SELECT * FROM robots WHERE SUBSTR(location, -2) = 'NY';

5

Resultado:



6

Interpretación:
La consulta toma las dos últimas letras de cada ubicación, que corresponden a la abreviatura del estado, y conserva solo las que dicen NY. Seattle, Palo Alto, Tampa y Spring Valley... no, Spring Valley sí está en NY; los que quedan fuera son Palo Alto (CA), Tampa (FL) y Seattle (WA).

7

Aprendizaje:
SUBSTR con índice negativo es útil cuando lo que buscas está al final de un texto y su longitud varía. Además, la lección muestra que el mismo filtro se puede hacer con LIKE, por ejemplo WHERE location LIKE '%, NY', que también funciona aquí.

8

Aplicación en el mundo real:
En seguridad, SUBSTR permite extraer partes de identificadores o registros, por ejemplo el prefijo de una IP para agrupar equipos por subred (SUBSTR(ip, 1, 10)), los últimos caracteres de un hash para comparar archivos, o el código de país de un número de teléfono registrado en una cuenta.

1

Ejercicio 31: COALESCE (primer valor no nulo de una lista de columnas)

2

Objetivo del ejercicio:
Obtener el arma principal de cada combatiente, con prioridad: tanque, luego pistola y por último espada.

3

Concepto SQL:
COALESCE(col1, col2, col3): devuelve el valor de la primera columna que no sea NULL. Si todas son NULL, devuelve NULL.

4

Consulta ejecutada:
SELECT name, COALESCE(tank, gun, sword) AS weapon FROM fighters;

5

Resultado:



6	**Interpretación:** US Marine tiene tanque, así que ese es su arma. Booth no tiene tanque, así que se toma la pistola. Zorro no tiene tanque ni pistola, así que se toma la espada. Innocent Bystander no tiene nada, así que el resultado es NULL.
7	**Aprendizaje:** El orden de los argumentos define la prioridad: lo que va primero se revisa primero. Si cambiaras el orden a COALESCE(sword, gun, tank), Marine usaría su espada en lugar de su tanque. Además, COALESCE puede usar un valor por defecto al final, como COALESCE(tank, gun, sword, 'Sin arma'), para evitar NULL.
8	**Aplicación en el mundo real:** En seguridad, COALESCE sirve para tomar el primer dato disponible de varias fuentes, por ejemplo la fecha del último inicio de sesión de una cuenta, o si no existe, la fecha de creación. También sirve para definir una prioridad al calcular el responsable de un activo: primero el dueño asignado, luego el área, y por último "sin responsable".

CONCLUSIONES

Los 31 ejercicios me llevaron de lo más básico, como SELECT * y WHERE, hasta consultas con varias tablas, subconsultas, valores nulos, fechas, CASE y funciones de texto como SUBSTR y COALESCE. Lo que más me ayudó fue ver cómo cada concepto se apoya en el anterior: primero aprendí a filtrar, luego a agrupar y resumir, y al final a unir tablas y dar formato al resultado.

Lo que más me costó fue entender que los detalles cambian el resultado. Por ejemplo, en el ejercicio 23 escribí = cuando el enunciado pedía <>, y eso me devolvió a Barney y Lily en lugar de solo a Doogie. En los ejercicios de LEFT JOIN aprendí que el tipo de unión decide qué filas sobreviven, y con NULL que no se compara con =. Me quedó claro que leer bien el enunciado es tan importante como escribir bien la consulta.

También noté que varias cosas que se aprenden aquí, como usar alias, filtrar con LIKE o CASE, se pueden aplicar a trabajo real de seguridad de la información y auditoría: revisar cuentas sin MFA, detectar datos faltantes, priorizar hallazgos o cruzar tablas de usuarios y accesos.

Y una reflexión honesta: Algunas de mis respuestas en el camino tuvieron errores, como el cuadro del ejercicio 23 o la interpretación confusa del 30. Eso me enseñó a verificar contra el resultado esperado y no a dar por buena la primera respuesta. Mi conclusión final es que SQL no es memorizar sintaxis, sino entender qué filas quiero, de qué tabla vienen y qué condiciones deben cumplirse.